

Modelo de Amenazas - IMS Rapanui

Owner: Domingo Ruiz-Tagle
Reviewer: Cristián Rojas
Contributors:
Date Generated: Wed May 27 2026

Executive Summary

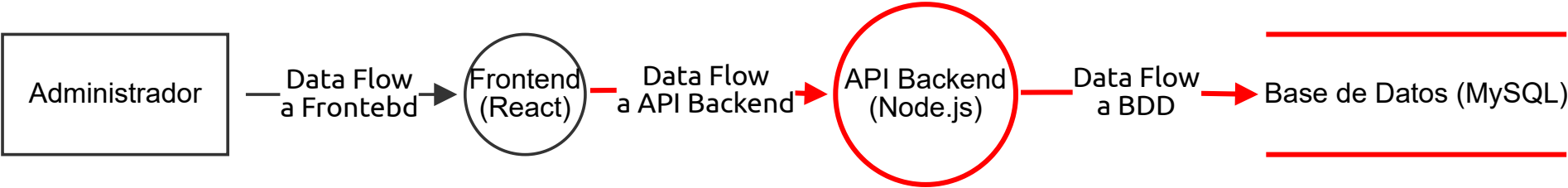
High level system description

Sistema de inventario full-stack (Frontend en React, Backend en Node.js/Express, Base de Datos MySQL).

Summary

Total Threats	5
Total Mitigated	0
Total Open	5
Open / Critical Severity	1
Open / High Severity	3
Open / Medium Severity	1
Open / Low Severity	0

DFD - IMS Rapanui



DFD - IMS Rapanui

Administrador (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Frontend (React) (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

API Backend (Node.js) (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Bypass del Frontend — CRUD no autorizado	Elevation of privilege	High	Open	5.75	Un atacante puede descubrir la URL directa de la API en Node.js y enviar peticiones POST o DELETE maliciosas saltándose el frontend en React, eliminando o alterando libros del inventario directamente.	Implementar validación estricta de tokens JWT en cada ruta de Express.js para que solo usuarios autenticados ejecuten acciones CRUD.
2	DoS al endpoint de la calculadora	Denial of service	High	Open	4	Un atacante puede enviar miles de peticiones simultáneas al endpoint de cálculo en Express.js, saturando el servidor Node.js y dejando el sistema de inventario completamente inoperativo.	Implementar Rate Limiting en Express.js para bloquear IPs que realicen demasiadas solicitudes en un corto período de tiempo (ej: express-rate-limit).

Base de Datos (MySQL) (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
3	Inyección SQL, Manipulación de MySQL	Tampering	Critical	Open	7.8	Si Node.js concatena directamente los datos del usuario en las consultas a MySQL sin sanitizar, un atacante puede inyectar comandos SQL para eliminar tablas completas del inventario o alterar precios de libros.	Usar siempre consultas parametrizadas o un ORM como Sequelize en el backend para interactuar con MySQL, evitando la ejecución de código arbitrario.

Data Flow a Frontebd (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Data Flow a BDD (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	Manipulación de peso/precio	Tampering	Medium	Open	3.8	Si la validación de peso/precio ocurre solo en el frontend, un usuario malintencionado puede interceptar y modificar la petición antes de que llegue al servidor, alterando el cobro final del pedido.	Toda la validación matemática y lógica de negocio de pesos y precios debe re-validarse obligatoriamente en el backend (Node.js) antes de procesar cualquier transacción.

Data Flow a API Backend (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	Sniffing de credenciales HTTP	Information disclosure	High	Open	5.5	Si la comunicación entre el Frontend (React) y el Backend (Node.js/Express) usa HTTP plano sin cifrado, un atacante en la misma red Wi-Fi puede interceptar el tráfico y robar datos de sesión o credenciales de usuario.	Implementar y forzar HTTPS/TLS con certificados válidos en toda la comunicación entre cliente y servidor.